

Memory Efficient Doubly Linked List –Operation Understanding

1. Insert At Beginning

```
newNode->npx = XOR(nullptr, head);
```

This exhibit the identity property : $X \oplus 0 = X$.

It stores the address stored in head ,the address of first node : A.

When List is not Empty:

```
if (head != nullptr)
{
    Node *next = XOR(nullptr, head->npx);
    head->npx = XOR(newNode, next);
}
```

Here we need a pointer next to hold the identity property : $X \oplus 0 = X$. Here X is $head \rightarrow npx$, this is current address that becomes node B, Actually it is :

The next pointer of A is : $NULL \oplus B$

$head \rightarrow npx = XOR(newNode, next)$.

It maintains the property : Next pointer of B is: $A \oplus C$.

That is the first address where data and npx is stored,

the npx pointer will store the XOR result : $XOR(newNode, next)$

i. e. $XOR(newNode: Node A, next: Node C)$

, This XOR result will help to traverse to next node.

2. Insert At End

1. When List is Empty:

```
if (head == nullptr)
{
    newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
    head = newNode;
    cout << data << " inserted at the end." << endl;
    return;
}
```

Here: $newNode \rightarrow npx = XOR(nullptr, nullptr)$;

That is when there will be one node the npx pointer of the node will carry: nullptr.

2. When List is not Empty:

```
while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr){ break; }
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow \text{Address of Node C}$

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR(\text{Address of Node A}, \text{Address of Node C})$
The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: \text{Address of B}, curr \rightarrow npx: D)$

$\Rightarrow \text{Address of Node D}$

Note: D is the XOR result..

May not be address of Node D i.e. Result of $XOR(\text{Address of Node B}, \text{Address of Node D})$
The only time Node C's npx will contain Address of Node B when there will be only three node exists. Similarly Node D's npx will contain Address of Node C, when there will be only four node exist.

... .. *etc.*

Similarly ,when the loop fails when we get situation like:

$next = (prev: \text{Address of Node A}, curr \rightarrow npx: \text{Address of Node A}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: \text{Address of Node B}, curr \rightarrow npx: \text{Address of Node B}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: \text{Address of Node C}, curr \rightarrow npx: \text{Address of Node C}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... .. *etc.*

Again ,we have:

```
curr->npx = XOR(prev, newNode);  
newNode->npx = XOR(curr, nullptr);
```

Suppose we have 3 nodes in the list. Hence $curr \rightarrow npx$, indicates Node C's npx and Node D to be inserted at last , hence:

$curr \rightarrow npx = XOR(prev: \text{Address of B}, newNode: \text{Address of D});$

Node C's npx: Address of B $\oplus$ Address of D.

$newNode \rightarrow npx = XOR(curr: \text{Address of C} , NULL)$

i. e., Node D's npx : Address of C $\oplus$ NULL = Address of C.

Similarly,

Suppose we have 4 nodes in the list. Hence $curr \rightarrow npx$, indicates Node D's npx and Node E to be inserted at last, hence:

$curr \rightarrow npx = XOR(prev: Address\ of\ C, newNode: Address\ of\ E);$

Node D's npx: $Address\ of\ C \oplus Address\ of\ E$.

$newNode \rightarrow npx = XOR(curr: Address\ of\ D, NULL)$

i.e., Node E's npx : $Address\ of\ D \oplus NULL = Address\ of\ D$.

This is goes on ...

2. Insert At Position

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: Address\ of\ Node\ B)$

2. $next = XOR(prev: Address\ of\ A, curr \rightarrow npx: C)$

$\Rightarrow Address\ of\ Node\ C$

Note: C is the XOR result.

	$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node C i.e. Result of} \\ \text{XOR} \left(\begin{array}{l} \text{Address of Node A,} \\ \text{Address of Node C} \end{array} \right) \\ \text{The only time Node B's npx} \\ \text{will contain Address of Node A} \\ \text{When there will be only two node} \\ \text{exists.} \end{array} \right]$
--	---

3. $\text{next} = \text{XOR}(\text{prev: Address of B}, \text{curr} \rightarrow \text{npx: D})$

$\Rightarrow$ Address of Node D

Note: D is the XOR result..

	$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node D i.e. Result of} \\ \text{XOR} \left(\begin{array}{l} \text{Address of Node B,} \\ \text{Address of Node D} \end{array} \right) \\ \text{The only time Node C's npx} \\ \text{will contain Address of Node B} \\ \text{when there will be only three node} \\ \text{exists. Similarly Node D's npx will} \\ \text{contain Address of Node C, when there will} \\ \text{be only four node exist.} \end{array} \right]$
--	---

... etc.

Similarly, when the loop fails when we get situation like:

$\text{next} = (\text{prev: Address of Node A}, \text{curr} \rightarrow \text{npx: Address of Node A});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$\text{next} = (\text{prev: Address of Node B}, \text{curr} \rightarrow \text{npx: Address of Node B});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$\text{next} = (\text{prev: Address of Node C}, \text{curr} \rightarrow \text{npx: Address of Node C});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Again , we have:

```
next = XOR(prev, curr->npx);
```

If the `pos` goes from first to $n - 1$

Eg: $next = (prev: Address\ of\ B \oplus curr \rightarrow np\!x(np\!x\ of\ Node\ C): D),$

where, D is : $Address\ of\ B \oplus Address\ of\ D$, hence,

$\Rightarrow next = (Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

And , we know: by property $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

Hence, we get: $(Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

We get $next = Address\ of\ D$.

At n th (last node, say Node D)

$\Rightarrow next = (prev: Address\ of\ Node\ C \oplus curr \rightarrow np\!x(np\!x\ of\ Node\ D): D),$

where, D is : $Address\ of\ C \oplus NULL = Address\ of\ C$, hence,

$\Rightarrow next = XOR(prev: Address\ of\ Node\ C \oplus curr \rightarrow np\!x: Address\ of\ C)$

$\Rightarrow next = NULL$.

```
newNode->npx = XOR(curr, next);
```

If the `pos` goes from first to $n - 1$

Say, $\text{newNode} \rightarrow \text{npx} = \text{XOR}(\text{curr: Address of C, next: Address of Node D})$;

(As newNode will sit between Node C and Node D), basically :

Node D will become Node E and New Node becomes Node D,

as, it is following, the process:

The next pointer of D is: $C \oplus E$.

***i. e. NPX of New Node(Node D) will XOR result of Address of C,
and Address of Node E (which was Node D earlier).***

At nth (last node, say Node D)

$\text{newNode} \rightarrow \text{npx} = \text{XOR}(\text{curr: Address of D, next: NULL})$,

As, New Node will become Node E, hence,

The next pointer(NPX) of Node E (last Node) will be : $(D \oplus \text{NULL})$.

```
curr->npx = XOR(prev, newNode);
```

Now, say we need to enter a Node between Node C and Node D and we have series of nodes : Node A, Node B, Node C and Node D.

And after entering A new Node between Node C and Node D ,the New Node becomes Node D, and Node D becomes Node E.

Hence ,we need to update NPX (Next Pointer through XORing process) of Node C and Node E. Here ``curr → npx`` is the NPX of Node C.

Now why XORing address of prev and new Node? NewNode will become Node D and Prev hold address of Node B and we know:

The next pointer of C is : Address of Node B $\oplus$ Address of Node D

*curr → npx(NPX of Node C) = prev: Address of Node B $\oplus$
newNode: (Added Node which becomes Node D)Address of Node D.*

If $next \neq NULLPTR$, when we does not insert newNode to the last, newNode doesn't becomes last node.

```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

Suppose we want to enter after Node A:

$$next = (Address\ of\ A \oplus (Address\ of\ A \oplus Address\ of\ C))$$

$$next = Address\ of\ C$$

$$Nextnext = XOR(Curr: Address\ of\ B, next \rightarrow npx(NPX\ of\ NODE\ C): D)$$

$$Where, D = (Address\ of\ B \oplus Address\ of\ D)$$

$$\therefore Nextnext = Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D)$$

$$= Address\ of\ D$$

$$next \rightarrow npx = XOR\left(newNode: \begin{pmatrix} [Newly\ entered\ Node\ at\ Pos: B] \\ Address\ of\ newNode(Pos: B) \end{pmatrix} \oplus Address\ of\ D\right)$$

Here,

Next $\rightarrow$ NPX is Node C's NPX and the above is the exact update we needed.

Insert After Element

```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow$ Address of Node C

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR(\text{Address of Node A}, \text{Address of Node C})$
The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: \text{Address of B}, curr \rightarrow npx: D)$

$\Rightarrow$ Address of Node D

Note: D is the XOR result..

May not be address of Node D i.e. Result of $XOR(\text{Address of Node B}, \text{Address of Node D})$
The only time Node C's npx will contain Address of Node B when there will be only three node exists. Similarly Node D's npx will contain Address of Node C, when there will be only four node exist.

... etc.

Similarly ,when the loop fails when we get situation like:

$next = (prev: \text{Address of Node A}, curr \rightarrow npx: \text{Address of Node A}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: \text{Address of Node B}, curr \rightarrow npx: \text{Address of Node B}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: \text{Address of Node C}, curr \rightarrow npx: \text{Address of Node C}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Again ,we have:

```
next = XOR(prev, curr->npx);
```

If the `pos` goes from first to $n - 1$

Eg: $next = (prev: \text{Address of B} \oplus curr \rightarrow npx(\text{npx of Node C}): D),$

where, D is : $\text{Address of B} \oplus \text{Address of D}$, hence,

$\Rightarrow next = (\text{Address of B} \oplus (\text{Address of B} \oplus \text{Address of D}))$

And ,we know: by property $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

Hence,we get: $(\text{Address of B} \oplus (\text{Address of B} \oplus \text{Address of D}))$

We get $next = \text{Address of D}$.

At nth (last node, say Node D)

$\Rightarrow next = (prev: Address\ of\ Node\ C \oplus curr \rightarrow npx(npx\ of\ Node\ D): D),$

where, D is : $Address\ of\ C \oplus NULL = Address\ of\ C$, hence,

$\Rightarrow next = XOR(prev: Address\ of\ Node\ C \oplus curr \rightarrow npx: Address\ of\ C)$

$\Rightarrow next = NULL.$

```
newNode->npx = XOR(curr, next);
```

If the `pos` goes from first to $n - 1$

Say, $newNode \rightarrow npx = XOR(curr: Address\ of\ C, next: Address\ of\ Node\ D);$

(As $newNode$ will sit between Node C and Node D), basically :

Node D will become Node E and New Node becomes Node D,

as, it is following, the process:

The next pointer of D is: $C \oplus E$.

i. e. NPX of New Node(Node D) will XOR result of Address of C, and Address of Node E (which was Node D earlier).

At nth (last node, say Node D)

$newNode \rightarrow npx = XOR(curr: Address\ of\ D, next: NULL),$

As, New Node will become Node E, hence,

The next pointer(NPX) of Node E (last Node) will be : $(D \oplus NULL).$

```
curr->npx = XOR(prev, newNode);
```

Now, say we need to enter a Node between Node C and Node D and we have series of nodes : Node A, Node B, Node C and Node D.

And after entering A new Node between Node C and Node D , the New Node becomes Node D, and Node D becomes Node E.

Hence , we need to update NPX (Next Pointer through XORing process) of Node C and Node E. Here ``curr → npx`` is the NPX of Node C.

Now why XORing address of prev and new Node? NewNode will become Node D and Prev hold address of Node B and we know:

The next pointer of C is : Address of Node B $\oplus$ Address of Node D

*curr → npx(NPX of Node C) = prev: Address of Node B $\oplus$
newNode: (Added Node which becomes Node D)Address of Node D.*

If $next \neq NULLPTR$, when we does not insert newNode to the last, newNode doesn't becomes last node.

```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

Suppose we want to enter after Node A:

$$next = (Address\ of\ A \oplus (Address\ of\ A \oplus Address\ of\ C))$$

$$next = Address\ of\ C$$

$$Nextnext = XOR(Curr: Address\ of\ B, next \rightarrow npx(NPX\ of\ NODE\ C): D)$$

$$Where, D = (Address\ of\ B \oplus Address\ of\ D)$$

$$\therefore Nextnext = Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D)$$

$$= Address\ of\ D$$

$$next \rightarrow npx = XOR\left(newNode: \begin{pmatrix} [Newly\ entered\ Node\ at\ Pos: B] \\ Address\ of\ newNode(Pos: B) \end{pmatrix} \oplus Address\ of\ D\right)$$

Here,

Next $\rightarrow$ NPX is Node C's NPX and the above is the exact update we needed.
